Laboratorio 5.2: Recomposición y State Hoisting

1. Contexto y Objetivos

Entender el Árbol de Composición y el ciclo de vida de un Composable es fundamental para construir aplicaciones móviles de alto rendimiento en 2026. En este laboratorio, los estudiantes desarrollarán un **Simulador de Domótica** (Luces Inteligentes). Este ejercicio permite visualizar físicamente el proceso de **Recomposición Inteligente** y aplicar el patrón de **State Hoisting (Elevación de Estado)**, asegurando una "Fuente única de verdad".

Objetivos de Aprendizaje:

- Implementar el patrón *State Hoisting* para separar la lógica de estado de la representación visual.
- Monitorear y analizar el proceso de Recomposición e Identificar el fenómeno de *Skipping* (Omisión inteligente).
- Consolidar el uso de modificadores y contenedores básicos en un entorno API 36.

2. Requisitos Previos

- Contar con la **Plantilla Maestra** del Laboratorio 1 configurada (Java 21, Kotlin 2.0, API 36).
- Haber completado el Laboratorio 5.1 sobre estados básicos.

3. Descripción General

Se construirá una aplicación con una jerarquía de dos niveles:

- **Padre (SmartHome):** Gestiona el estado de la luz (encendido/apagado).
- **Hijo (LightBulb):** Un componente visual "tonto" que solo recibe órdenes y se dibuja según el estado.

Se utilizarán etiquetas de registro (Logs) para observar en tiempo real qué funciones se vuelven a ejecutar y cuáles son omitidas por el framework para ahorrar batería y CPU.

4. Paso a Paso Detallado

Paso 1: Clonación y Preparación (Continuidad Técnica)

1. Cierre cualquier proyecto abierto en **Android Studio Ladybug**.
2. En el explorador de archivos, copie la carpeta de su proyecto anterior (Lab 5.1) y renómbrela a: Lab5.2_NombreApellido.
3. Elimine las carpetas ocultas .gradle, .idea y build para limpiar la caché.
4. Abra la nueva carpeta en Android Studio.
5. Vaya a settings.gradle.kts y cambie la línea a: rootProject.name = "SmartHomeSim".
6. Pulse el botón **Sync Project with Gradle Files** (icono del elefante).

Paso 2: Implementación de la Jerarquía de Componentes

Abra el archivo: app/src/main/java/com/example/myapplication/MainActivity.kt.
Borre el contenido previo y reemplace con la siguiente estructura base de ingeniería:

```
package com.example.myapplication

import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Lightbulb
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface(color = MaterialTheme.colorScheme.background) {
                    SmartHome()
                }
            }
        }
    }
}

@Composable
fun SmartHome() {
    // Trazabilidad: Log para observar recomposición del Padre
    Log.d("RECOMPOSE", "DIBUJANDO PADRE (SmartHome)")

    // Estado Hoisting: El padre posee la verdad
    var isLightOn by remember { mutableStateOf(false) }

    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(text = "Centro de Control Domótico", style =
MaterialTheme.typography.headlineMedium)

        Spacer(modifier = Modifier.height(32.dp))

        // Enviamos el estado AL HIJO y una lambda para recibir cambios
```

```
        LightBulb(isOn = isLightOn)

        Spacer(modifier = Modifier.height(32.dp))

        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Interruptor Principal")
            Switch(
                checked = isLightOn,
                onCheckedChange = { isLightOn = it }
            )
        }
    }
}

@Composable
fun LightBulb(isOn: Boolean) {
    // Trazabilidad: Log para observar recomposición del Hijo
    Log.d("RECOMPOSE", "DIBUJANDO HIJO (LightBulb)")

    Icon(
        imageVector = Icons.Default.Lightbulb,
        contentDescription = "Bombilla",
        modifier = Modifier.size(120.dp),
        tint = if (isOn) Color.Yellow else Color.Gray
    )
}
```

Paso 3: Verificación Intermedia y Logcat

1. Ejecute la aplicación en su emulador **API 36**.
2. En Android Studio, abra la pestaña **Logcat** (en la parte inferior).
3. En el cuadro de búsqueda del Logcat, escriba tag:RECOMPOSE.
4. Mueva el interruptor (Switch). Observe cómo ambos mensajes aparecen cada vez que cambia el estado. *Análisis: Como el hijo depende directamente de 'isLightOn', Compose debe redibujarlo*.

5. Resultados Esperados

Al interactuar con la app, la bombilla debe cambiar de Gris a Amarillo instantáneamente. En el Logcat se debe evidenciar que tanto el contenedor SmartHome como LightBulb se recomponen al detectar el cambio de estado en la variable isLightOn.

6. Desafío

Reto: Añada un botón de "Refrescar UI" que cambie una variable de estado independiente y observe por qué la bombilla NO se redibuja (Skipping).
Resolución Técnica: Plantee la solución al desafío

7. Entregables

1. Informe **PDF** con capturas de pantalla de:
    - Captura del Logcat filtrado demostrando el flujo de recomposición.
    - Captura que evidencie la resolución del Desafío.
    - Copia del código como Anexo (**MainActivity.kt**).
    - Respuestas a preguntas de haberlas.
    - Explique por qué el componente Hijo se redibuja cuando movemos el Switch del Padre basándose en el concepto de "UI como función del estado".
    - Demás puntos que debe contener el informe como conclusiones, recomendaciones, etc.
2. **Enlace al repositorio de GitHub** con el commit final.

8. Criterios de Evaluación

No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe